國立臺灣大學電機資訊學院資訊網路與多媒體研究所

碩士論文

Graduate Institute of Networking and Multimedia
College of Electrical Engineering and Computer Science
National Taiwan University

Master Thesis

多重觸控桌面顯示系統之手指偵測

Finger Detection for Multi-Touch Tabletop Display System

莊舒婷

Su-ting, Chuang

指導教授：洪一平 博士

Advisor: Yi-Ping Hung, Ph.D.

中華民國 99 年 7 月

July, 2010

誌謝

能完成這篇論文，我要特別感謝我的指導教授洪一平老師，不論是在研究領域還是人生的方向，老師總是能適時的指引我走向正確的道路，讓我在這兩年的研究生活中成長了許多。

接著我要感謝在這段做研究的時間內，曾帶領過我的 Muscle、春神、偉哥，謝謝你們不辭辛勞的在研究上給我許多建議，在我遭逢研究瓶頸的時候指點我，另外，我要特別感謝這兩年中一起奮鬥的慧姍、佩君、510、54way、小鬍子、瑞佑、柏訊，這段時間內因為你們的陪伴，研究生活中充滿了歡笑。另外，還要感謝實驗室的各個學長姊學弟妹，因為你們，實驗室成為一個既溫馨又歡樂的地方。

最後，要感謝我的家人及好朋友們，你們的關心和祝福是促使我不斷的前進的動力。

中文摘要

隨著顯示科技及輸入技術的發展，將電腦引入日常生活的互動桌面系統(tabletop)越來越受到注目，許多研究致力於如何建構大型的互動顯示系統，基於桌面系統的需求，利用紅外線攝影機及紅外線投燈器，透用影像處理的方式來進行手指偵測及追蹤的系統日益增加。然而，基於電腦視覺技術開發的多重觸控手指偵測系統會受到紅外線投燈器照射光分佈不均勻的影響，導致多指觸控偵測效果不如預期；另外，現存多指觸控偵測系統在效能上亦不如預期，進而影響指觸人機互動系統的操作性；再者，對一個基於影像處理的方式開發的手指偵測系統而言，為了達到良好的操作性，有許多相關的參數需要調整，對使用者而言，是相當煩雜且耗時的工作。

因此， 在本研究中我們開發了一個運用圖形處理器加速的高效能多重觸控手指偵測系統，不僅解決了紅外線投燈器照射光分佈不均勻的問題，還提出一個基於取樣搜尋的方法，協助使用者決定適用於多指觸控系統的參數。

關鍵字：手指偵測 (Finger Detection), 多重觸控 (Multi-Touch), 互動桌面 (Interactive Tabletop) , 加速 (Acceleration)

ABSTRACT

Advances in display and input technologies either based on electronic devices or computer vision have led to booming popularity of direct-touch interactive surfaces. Since there is a trend to use larger display, computer vision approach is adopted for direct-touch interaction due to low cost. However it suffered from non-uniform lighting problem. Besides, the computation efficiency is not good enough in existent multi-touch detection systems. Furthermore, determining parameters for detection system includes try-and-error process which is tedious and time-consuming. Therefore, we modified and designed a finger detection system based on previous work to improve drawbacks mentioned above. In this work, we develop a GPU-accelerated multi-touch detection which solves the problem of non-uniform lighting condition of tabletop system. In addition, we propose a sampling-based parameter search method to help user determine an applicable set of parameters for multi-touch detection system.

Keywords—Finger Detection, Multi-Touch, Interactive Tabletop System, Acceleration

CONTENTS

口試委員會審定書 .....#

誌謝 i

中文摘要 ii

ABSTRACT iii

CONTENTS iv

LIST OF FIGURES vi

LIST OF TABLES viii

Chapter 1 Introduction 1

Chapter 2 Related Work 3

2.1 Hardware Design 3

2.2 Finger Detection 7

Chapter 3 System and Method 9

3.1 Hardware Configuration 9

3.1.1 Touch-sensing Mechanism 10

3.1.2 Display Mechanism 12

3.2 Software Implementation 12

3.2.1 Image Stitching 14

3.2.2 Finger Detection 22

3.2.3 Finger Tracking 27

3.2.4 Parameter Determination 28

Chapter 4 Experiments 33

Chapter 5 Conclusion and Future Work 42

5.1 Conclusion 42

5.2 Future Work 42

Bibliography 44

LIST OF FIGURES

Figure 2.1: The DiamondTouch system [1]. 4

Figure 2.2. Two computer-vision-based approaches. 5

Figure 2.3: The *Microsoft Surface* tabletop system [7]. 6

Figure 2.4: The Reactble [8]. 6

Figure 3.1: Hardware Configuration of our system. 9

Figure 3.2: Order of diffuser layer and touch-glass layer. 10

Figure 3.3: The flowchart of our multi-touch detection system 13

Figure 3.4: The architecture of our multi-touch finger detection system. 13

Figure 3.5: The flowchart of Image Stitching 15

Figure 3.6: The difference of applying *Undistortion* or not. 17

Figure 3.7: Different size of fingertip among various positions on table. 18

Figure 3.8: Homography calculation 19

Figure 3.9: Mapping among camera coordinate and virtual space. 19

Figure 3.10: The flowchart of Image fusion. 21

Figure 3.11: The flowchart of finger detection algorithm in TouchLib 22

Figure 3.12: The flowchart of finger detection algorithm in our system. 22

Figure 3.13: The difference between DoG and Simple Highpass 25

Figure 3.14: The images produced after processing each procedure 27

Figure 3.15: The schematic of optimizing parameters 30

Figure 3.16: Explanation for occurrence of miss and false alarm. 31

Figure 3.17: Explanation for occurrence of errors and the sketch of uncontinuity 31

Figure 3.18: Explanation of occurrence of false alarm due to multiple detected fingers 32

Figure 4.1: Performance evaluation of multi-touch detection system. 34

Figure 4.2: The schematic view of particle filtering algorithm. 38

Figure 4.3: The diagram of particle sampling applied on our system. 41

LIST OF TABLES

Table 4.1: Range and step for each parameter in the experiment.....37

Chapter 1

Introduction

Advance in input technologies that allows users to interact with computers in a more intuitive way has gained more and more attentions. Multi-touch is one of the most attracting human-computer interaction techniques in recent years. Unlike single point touch-screens such as mouse, and track-ball, multi-touch interface allows users to manipulate graphic objects with multiple fingers at the same time. Considering the trend of utilizing large display, the booming popularity of multi-touch has led to expansions of various vision-based multi-touch techniques [7][8][9][14][15].

In previous work, the well-known and most generally used finger detection system which adopts computer vision technologies is TouchLib, an open source multi-touch package. The detection performance is stable in most case. But it suffered from non-uniform lighting problem and computational efficiency. In fact, non-uniform lighting is a common issue while adopting vision-based multi-touch detection system because it will affect finger recognition result of a multi-touch detection system. This problem becomes serious owing to the preference of exploiting large display nowadays. In addition, computational efficacy is an essential concern for a multi-touch detection system. If the detection rate is low, delay between developer’s operation and reaction from interactive application will occur which may frustrate user making them consider that the detection system doesn’t work well.

Furthermore, current multi-touch techniques based on computer vision are built by concatenating a series of basic image processing filters in order to reach real-time performance. These filters usually contain parameters that require multi-touch developers to manually determine them in a manner of try-and-error process. The process could be time-consuming, the choice of parameters is inclined to touch characteristics of the developer responsible for adjusting the parameters of a multi-touch algorithm. Current multi-touch systems, however, do not support any evaluation process that helps developers to determine an applicable set of parameters, leading the resulting parameters tends to the touch characteristics of the developer.

In this work, we develop a GPU-accelerated multi-touch detection which solves the problem of non-uniform lighting condition of tabletop system. In addition, we propose a sampling-based parameter search method to help developer determine an applicable set of parameters for multi-touch detection system.

Chapter 2

Related Work

2.1 Hardware Design

Nowadays multi-touch surfaces are prevalent input devices for interactive user interface. DiamondTouch [1] , one of the earlier multi-touch systems allows multiple users to simultaneously interact on a tabletop. Through capacitive coupling, it associates touch regions with each user, useful for supporting user-specific operations. There are considerable technologies analogous to this kind of approach. Synaptics Touchpad [2][3] , which is extensively utilized(employed/exploit) in various devices, allows user to manipulate applications on laptops or hand-held devices through direct touch. In addition, Apple's iPhone [4] uses a touch-sensitive screen layered with capacitors arranged according to the coordinate in cell phone to support multi-touch. These capacitive-based sensing technologies can also detect finger contact, but they are limited to large sensing resolution due to connections among sensors and the high cost.

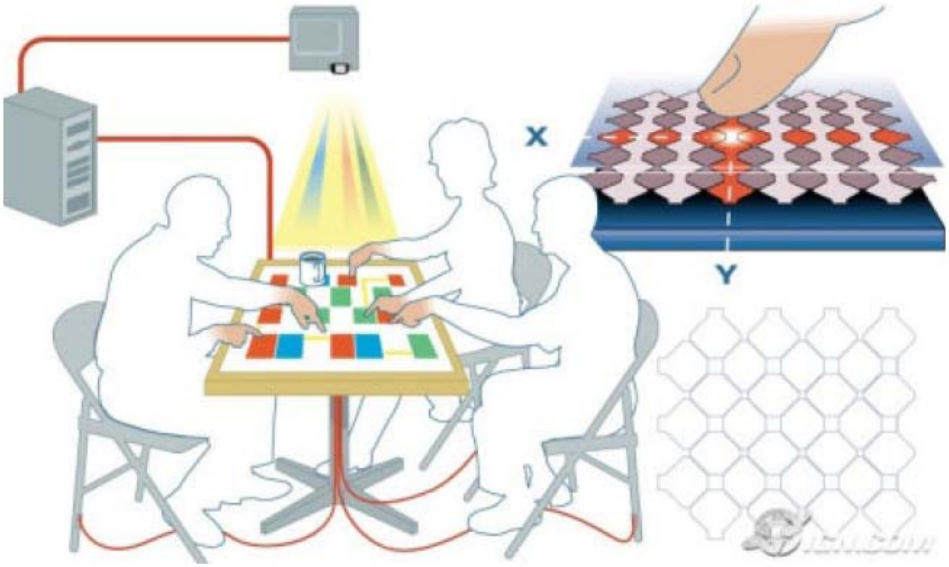
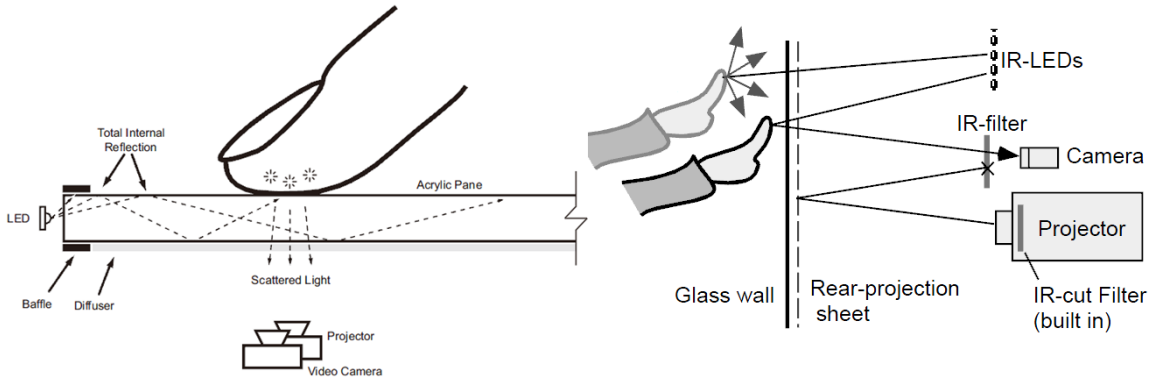


Figure 2.1: The DiamondTouch system [1].

Computer-vision-based technologies are also widely exploited to enable multi-touch surfaces. Diffused Illumination (DI) and Frustrated Total Internal Reflection (FTIR) are the two main technologies employed in multi-touch system. In FTIR [5] , which is proposed by Han, IR light is shined into the side of an acrylic panel. This light is trapped inside the acrylic by internal reflection. When a finger touches the acrylic surface, the light is “frustrated”, causing the light scattering downwards to where the IR camera is. Therefore, we can detect fingertips through the image containing scatter light which is picked up by the IR camera. However, the sensing image obtained from FTIR is clear only when users touch with little pressure on it.

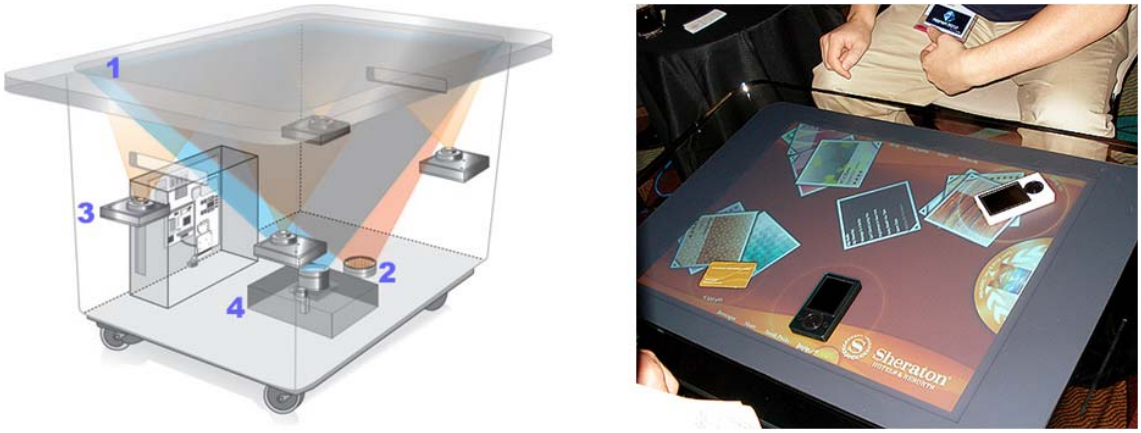


(a) The FTIR system (b) The DI system

Figure 2.2. Two computer-vision-based approaches.

(a) The schematic view of FTIR [5]. (b) The schematic view of DI [6].

An alternative technique of computer vision used to observe touch information is based on Diffused Illumination. In DI [6] , which is presented by Rekimoto and Matsushita, IR light is mounted underneath a hard clear surface with a diffuser. When an object touches the surface, it reflects more light than those in the behind and is sensed by a camera. This approach can not only detect the contact regions but also hovering interaction above the surface within a certain distance. In addition, it outperforms other techniques on its capability of object recognition and achieves touch-sensing for very large scale display with low cost. Microsoft Surface [7] , which is a 30-inch rear-projected multi-touch tabletop, has demonstrated this feature by attaching special coded markers under the bottom of objects. For example, when mobile phones are put on the surface, photos and business card related to the phone will be shown out by recognizing corresponding coded markers. The same function was also addressed in Reactable [8].



(a) The schematic view (b) Demo application

Figure 2.3: The *Microsoft Surface* tabletop system [7].

(a) The schematic view of *Surface*; the mechanism is based on DI which combines four IR cameras. (b) One demonstration on the Surface; cell-phone can be put the *Surface* and be recognized.

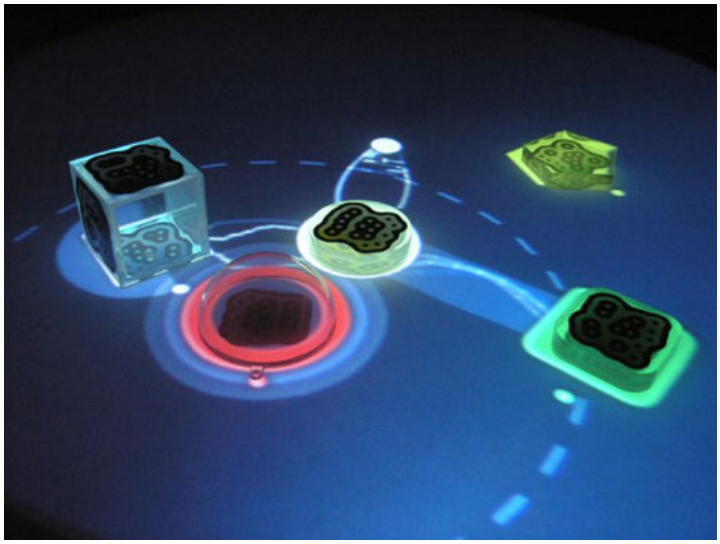


Figure 2.4: The Reactble [8].

Compared to capacitive-based sensing, vision-based systems provide higher fidelity in detecting the contact shape. In addition, cost of adopting DI on large scale tabletop is low. Therefore, we employ DI on our system.

2.2 Finger Detection

In recent years, we have witness an exponential growth of technologies to support multi-touch system. The following are various finger detection and finger tracking approaches. In [9], their work presented a robust finger tracking algorithm by shape analysis with a single camera. But whether the user's fingers touch the surface cannot be distinguished. In PlayAnywhere [10], Andrew D. Wilson proposed a method that examines the width of finger shadows shown on the surface to detect finger touching events. However, because the shadow may be occluded by other fingers or palms, this method only works when the hand with one finger spreading. Working with two cameras, TouchLight [11] proposed an effective fingertip finding approach by using stereo information. The system also can determine whether a given object is on the display or above it by analyzing stereo information. However, a special diffuser, HoloScreen, is required. Visual Touchpad [12][13], also using two cameras for hand tracking, performs multi-finger and whole-hand gestural interactions. The fingertips can be detected by contour analysis while the hand tracker provides positional information about the fingertips and finger orientations.

Along with a great deal of works and products of multi-touch proposed, the real need for a toolkit to help push forward the research of multi-touch HCI becomes evident. The DiamondTouch SDK, proposed by MERL in 2002, provided support for the development of applications that utilize DiamondTouch’s capabilities to implement computer supported collaboration and many more rich input modalities. Later in the DiamondSpin, the fundamental functionality that a multi-touched tabletop surface should embody was identified. The work presented a Java-based toolkit for efficiently

prototyping of and experimentation with multi-person interactive displays. It also provided a real-time polar to Cartesian transformation engine that has enabled around-the-table interaction as well as multiple work areas to be implemented. On the other hands, as the popularity and influence of multi-touch on small devices began to spread across the world, Apple released the iPhone SDK. It allows developers to create applications that leverage the iPhone’s groundbreaking multi-touch user interface.

Touchlib [15] is a library for creating multi-touch interaction surfaces. It handles tracking blobs of infrared light, and sends interactive application these multi-touch events, such as 'finger down', 'finger moved', and 'finger released'. Furthermore, Touchlib broadcast events in the TUIO protocol (which uses OSC). This makes it compatible with several other applications that support this protocol, such as Flash, PureData, and so on. Supporting the TUIO protocol also enables a distributed architecture where one machine can be devoted to detection and tracking and another machine can handle the application. The detail of their finger detection algorithm will be introduced in 3.2.2.

Chapter 3

System and Method

3.1 Hardware Configuration

Figure 3.1 gives a coarse view of our system. Under the displaying surface which includes a diffuser layer on top of a touch-glass layer, one projector, two IR cameras and several IR illuminators are installed. In the following, we will describe the hardware design in detail.

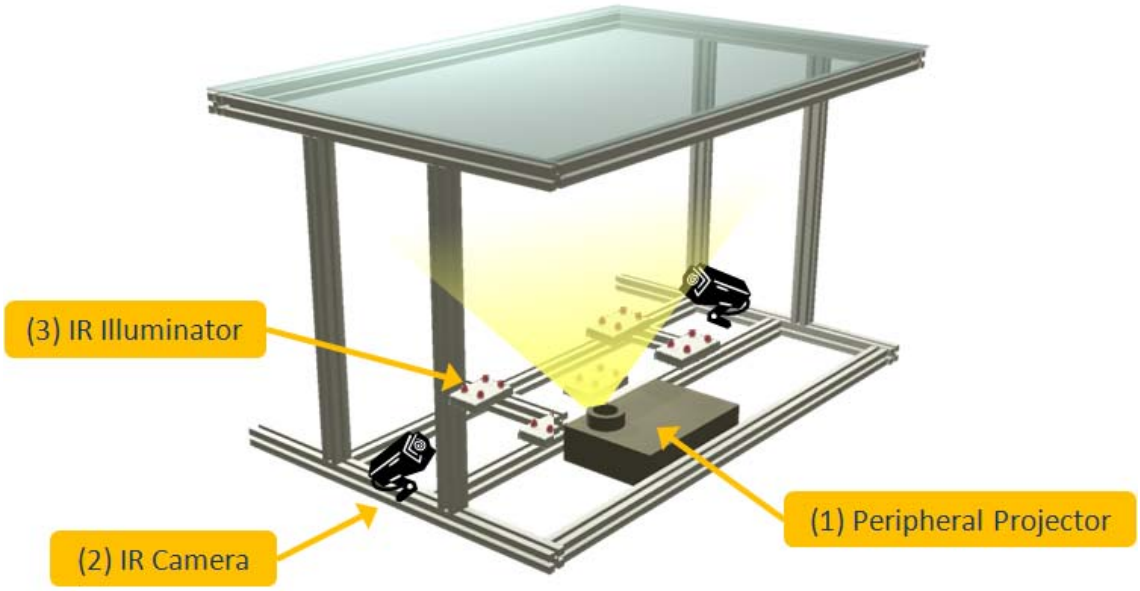


Figure 3.1: Hardware Configuration of our system

which includes: (1) a wide-angle peripheral projector, (2) two IR cameras, and (3) several IR LED lights.

3.1.1 Touch-sensing Mechanism

The approach for touch-sensing on our system is based on Diffused-lluminated (DI) setup. In this section, we will illustrate how we set up the surface, and how to install IR illuminators and IR cameras in our system to fulfill touch-sensing purpose.

The surface of this kind of tabletop system is usually composed of a diffuser layer and a touch-glass layer. The diffuser layer is placed directly above or beneath the touch-glass layer. However, because the touch-glass layer is a reflective surface, the relative placement of these two layers can lead to distinct problems in touch detection and screen projection from above the surface. In the following, we discuss the advantages and disadvantages about the relative placement of the diffuser and touch-glass layers.

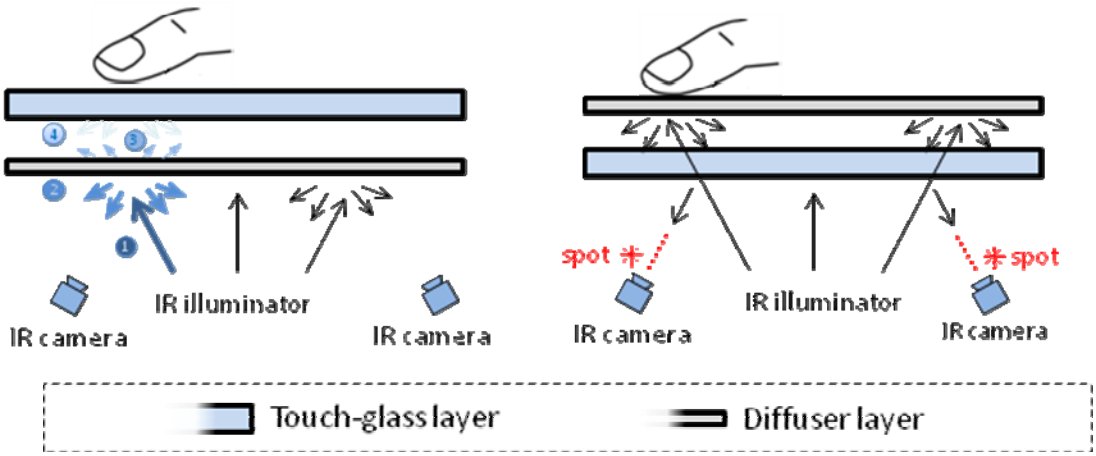


Figure 3.2: Order of diffuser layer and touch-glass layer.

Two common relative placement cases of the diffuser and touch-glass layers for a DI tabletop system

In the case of the touch-glass above the diffuser layer, the IR reflection cause by

touched finger will be degraded and the projected contents of applications won't be shown directly beneath finger. As shown in IR light will be diffused twice while passing through diffuser layer. The first time happened in passing from IR illuminator to touched finger and the second time occurs in reflecting from touched finger to IR camera. Under the circumstance, the IR reflection of touched finger captured by IR camera will be degraded and lead to a deteriorated finger touch response. As a result, it's hard to have a obvious direct-touch response compare to hover gesture which will make touch-sensing difficult. In addition, since contents of applications will present on diffuser layer, the installation of diffuser layer under touch-glass will let users feel contents is in a certain distance, depends on the thickness of the touch-glass, from their finger instead of directly beneath their finger.

In the case of the diffuser layer above the touch-glass layer, we are able to obtain the better quality of direct-touch response and make contents of application a preferable vision. However, the touch-glass causes reflections of the projection and IR rays from beneath the table. The luminance degradation due to reflection of the projection is acceptable, because the projector provides luminance far greater than required. The reflections of IR rays, however, cause an IR spot region in each camera view, resulting in dead zones in the image processing.

In our implementation, we placed the diffuser layer on top of the touch-glass layer in order to obtain the best quality projections both from beneath the table. The problem of IR spots can be solved by using two IR cameras. To overcome the IR spot effect, we use two IR cameras installed at two corners of the table, both capturing the tabletop. Because the two cameras observe the table surface from different angles, the IR spot reveals a disjointed region of the tabletop in each camera view (Fig. 4). By masking out the IR spot regions and blending together the two views, we can remove the IR spot

effect.

In order to support multi-touch on the coverage of entire table and to overcome IR spot effect mentioned above, we use two identical IR cameras in our system, each camera capturing half of the tabletop. Furthermore, there is certain overlapping on these two camera view which is not only for solving the dead zone problem but also for generating a correspondence between cameras. Moreover, several IR LED lights are mounted under the table, to provide IR illumination.

Since the displaying surface is semi-opaque and diffusive, the user's hand or any other objects in a distance away from the surface cause no IR reflection and are invisible to the cameras. When fingers get closer to the surface, more IR light will be reflected to the cameras and touch information can be recognized.

3.1.2 Display Mechanism

The image outputs of our system are displayed on the tabletop surface through one DLP projectors, which is used for the peripheral region (Figure 3.1(1)). The yielding resolution of this peripheral projector is about 27 x 23 ppi with output image size of 1280 x 720 pixels and the table size of 47 x 32 inches.

3.2 Software Implementation

Our multi-touch detection system can be divided into three parts - image stitching, finger detection, and finger tracking (Figure 3.3). Image stitching combines multiple camera view into a virtual camera view for the sake of adopting large scale surface and removing IR spot effect mentioned before. The data fusion in the overlapped area is carefully handled. Finger detection recognizes fingertips touched on surface. Finger

tracking smoothes the trajectory of finger.

Figure 3.3: The flowchart of our multi-touch detection system

Our multi-touch finger detection system is developed based on concept of filter. The advantage is that since each component of our finger detection system is developed as a filter, we can tray whatever combination of them to achieve optimal performance. This is useful especially for image processing part of recognizing fingers. In addition, since we utilize IR cameras to capture image of tabletop, our system should be able to deal streaming data well. Hence, we develop our multi-touch detection system based on DirectShow[18] which is an extensible, filter-based framework that provides high-quality capture and playback of multimedia streams. For the sake of speeding up, most part of our multi-touch system is developed based on GShow which is a GPU-accelerated framework composed of DirectX and DirectShow.

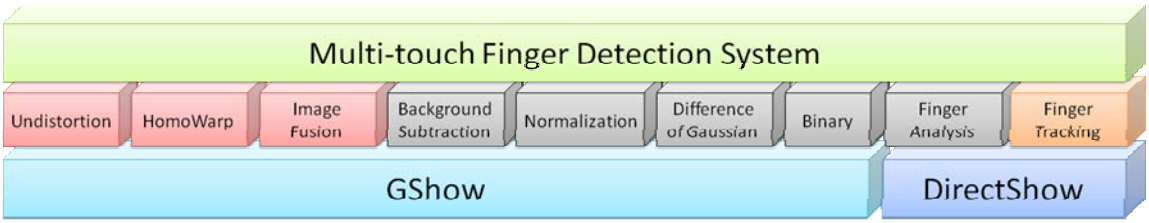


Figure 3.4: The architecture of our multi-touch finger detection system.

GShow and DirectShow are base frameworks of our implementation. The cubes on top of them are image processing filters which compose our multi-touch finger detection system.

3.2.1 Image Stitching

In this section, we will introduce the reasons why we need to exploit image stitching, and explain how we achieve this goal subsequently.

In order to fulfill the preference of exploiting large display, we have to employ multiple cameras to support multi-touch on the coverage of entire table. A large quantity of existing detection modules are adopted by using single camera. Therefore, user needs to add an additional component for dealing with finger detection result from multiple cameras. To be specific, it is required for user to match fingers after getting positions and IDs from each camera. Hence, for the sake of being compatible with existing detection module facilely, they need to expend extra effort.

Considering compatibility with existing detection module, we adopt an approach of making multiple camera views into a virtual camera view. Thus, no matter how many cameras are employed, it is regarded as a singer camera input for each detection system. In addition, the approach can also help us remove IR spot effect mentioned above. Furthermore, since we implementing this approach based on GPU, there doesn't exist inefficient problem in computation.

Under this circumstance, we do several pre-processing on images captured by each camera and stitch them into a large image to make it look like a real tabletop, named virtual camera by us.

Image stitching includes four procedures – capture images from IR cameras, undistort images, warp images, stitch images (Figure 3.5). For the convenience of describing, we name the procedure of undistorting images "*Undistortion*". In addition, since we warp images by homography, we denominated it "*HomoWarp*" for short. The flowchart is shown in the Figure 3.5. We illustrate the intention of each procedure in the

following paragraph.

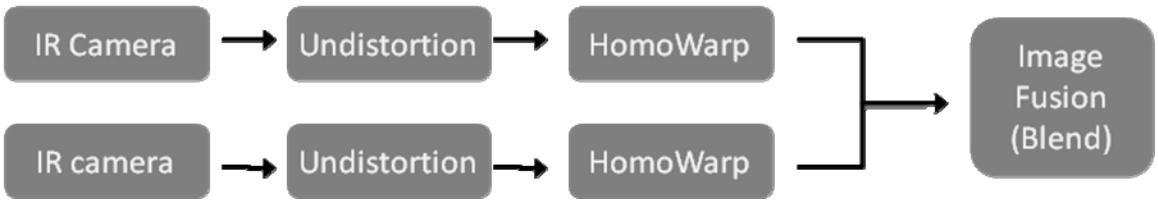


Figure 3.5: The flowchart of Image Stitching

(1) Undistortion

Our system uses wide-angle lens IR cameras which suffer from radial distortion. Although in the case of finger detection, distorted image won't affect detection result much. For future application, user may want do other foreground detection more than finger detection, for instance, marker recognition. In that case, distorted image may result in trouble while implementing recognition. For the sake of developing a general detection system which is compatible for various foreground recognition on tabletop, we utilize this procedure. On the other hand, *Undistortion* doesn't be employed while detecting fingertip in previous systems may due to computational efficiency. Since our system is implementing based on GPU, this concern is no longer important anymore.

Before it is possible to correct an image frame, it is required to obtain more information about the camera parameters. In order to calibrate the cameras, we need to present a known pattern from different angles and positions to cameras. In our case we used a chessboard pattern which is contains 9 x 7 crossed black and white squares. By presenting the pattern to the camera, we can recognize the pattern and assign coordinates to the intersections of the chessboard pattern. Furthermore, showing multiple views of the pattern from different angles can help us map all the coordinates in a three dimensional space. From the mapping of these points, we can calculate the

intrinsic parameters and the distortions coefficients.

Since the required parameters are known, we can now apply the image correction. The equation is:

$$\begin{aligned} x' &= x + x\left(k_1 r^2 + k_2 r^2\right) + \left(2p_1 xy + p_2\left(r^2 + 2x^2\right)\right) \\ y' &= y + y\left(k_1 r^2 + k_2 r^4\right) + \left(2p_1 xy + p_2\left(r^2 + 2y^2\right)\right) \end{aligned} \tag{3.1}$$

where $r^2 = x^2 + y^2$, x' and y' are the real (distorted) coordinates in the camera image, x and y are the ideal (distortion-free) coordinates in the camera image, k_1 and k_2 are the radial distortion coefficients, p_1 and p_2 are the tangential distortion coefficients. In this equation the second parts adds the radial distortion and the third part adds the tangential distortion. However, this equation is not complete. The center of the radial distortion is equal to the principle point. Therefore it is required to adjust the equation as follows:

$$\begin{aligned} u' &= u + \left(u - c_x\right)\left[k_1 r_2 + k_2 r^4 + 2p_1 y + p_2\left(\frac{r^2}{x} + 2x\right)\right] \\ v' &= v + \left(v - c_y\right)\left[k_1 r_2 + k_2 r^4 + 2p_1 x + p_2\left(\frac{r^2}{y} + 2y\right)\right] \end{aligned} \tag{3.2}$$

Where u' and v' are real observed (distorted) image coordinates in an ideal projection, u and v are true pixel image coordinates in an ideal projection, c_x, c_y are the coordinates of the principle point, k_1 and k_2 are the radial distortion coefficients, p_1 and p_2 are the tangential distortion coefficients.

As a result, we can use the relation from (3.2) to undistort captured images. The processing result is shown in Figure 3.6.

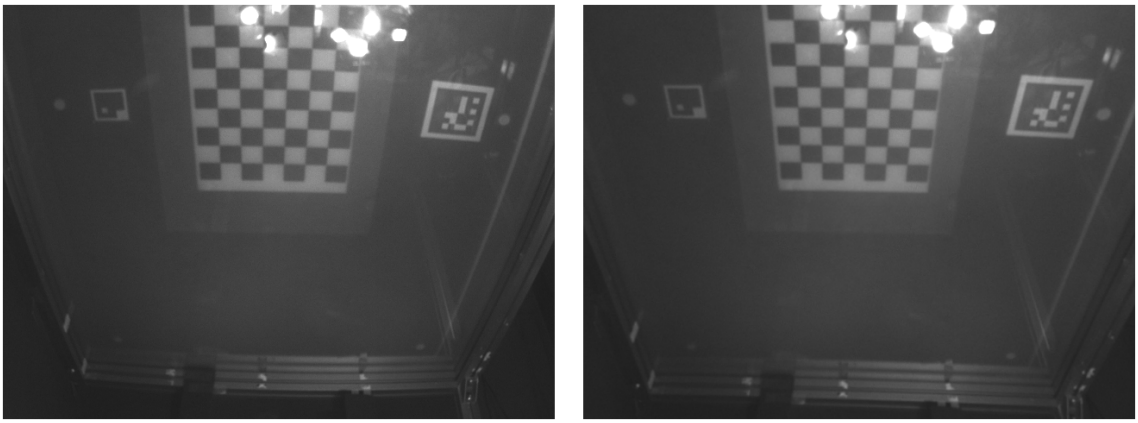


Figure 3.6: The difference of applying *Undistortion* or not.

The distorted image captured by our IR camera and undistorted image after employing *Undistortion* on it.

(2) Homowarp

We adopt two cameras in our system. In this procedure, we use homography to warp images of each cameras to a virtual space. There are two advantages to do this. First, we will have a uniform coordinate for different camera which represents virtual space instead of separate coordinate of camera spaces. Although coordinate transformation can be employed only on detected fingers instead of whole image, but there may exist problems when matching same fingers from different cameras. In addition, there is not necessary for us to worry about computational efficiency since we have speeded it up by implementing on GPU. Second, we can unify size of identical touched finger appearing at different position. The installation of cameras is not orthogonal to tabletop, in other words, there are included angle between camera view and surface, consequently fingertip near camera is larger than the fingertip far from camera (Figure 3.7). Because the way we determine whether the blob is a fingertip or not including examine the size of blob. Hence, if the size of fingertip varies from different position, there is not a standard measurement.

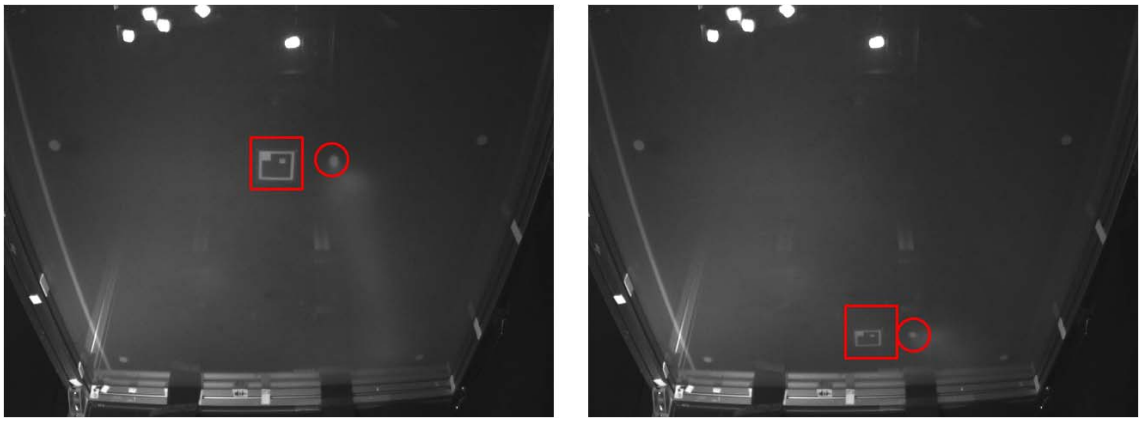


Figure 3.7: Different size of fingertip among various positions on table.

To make this phenomenon obvious, we put a tag to express this effect.

In this step, we calculate homography between cameras and virtual space in advance. We put six markers on the surface which located on left-top, center-top, right-top, left-bottom, center-bottom, and right-bottom of the tabletop. Corners of one camera correspond to corners of left virtual space; equally, corners of another camera correspond to corners of right virtual space (Figure 3.8). By given points correspondence, the homography matrix H of each camera can be calculated. After determined homography matrix, we apply them on input images to warp them into virtual space, shown in Figure 3.9.

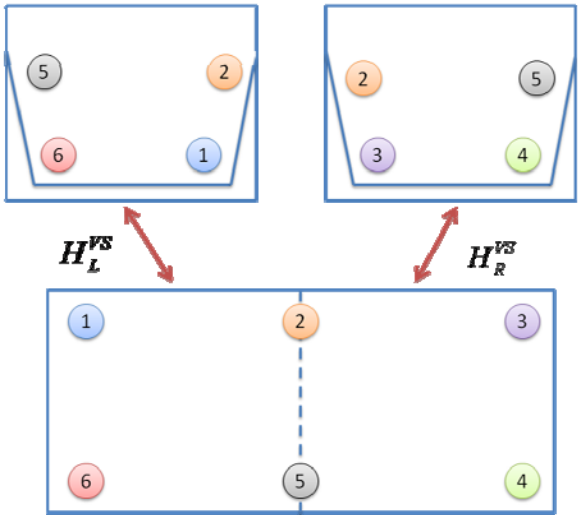


Figure 3.8: Homography calculation

It shows how we calculating Homography based on points' correspondence. Circles in same color are correspondent points.

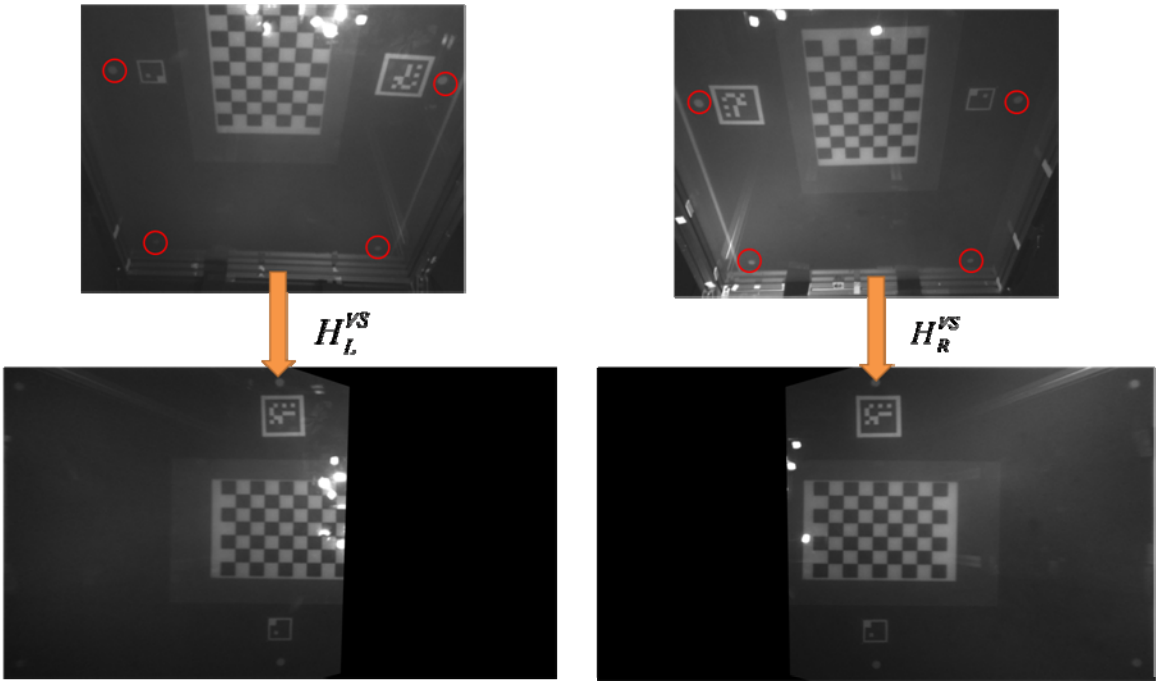


Figure 3.9: Mapping among camera coordinate and virtual space.

Source images maps to virtual space by employing *Homography*.

(3) Image Fusion

There are two reasons why we employ this procedure. First one is that, each of

camera view has a dead zone due to the IR spot effect, therefore, we employ this procedure to remove undesired position from one camera view and recover it from another camera view. Second one is that, we want to make vision of camera views become more intuitive for user. Because we adopt multi-camera, there will be more than one camera views which are separately present, hence we stitch them together to make a virtual camera view.

Image fusion includes masking images and stitching images together. After previous two procedures, camera views from each camera are already placed at right position of virtual space. However, they are separately shown on different image. In addition, there are dead zones in each image. Therefore, we need to decide which part of image we are going to employ. We fulfill this requirement by applying individual mask on each image. In this case, the dead zones are located on the opposite side of the camera view. In other word, dead zones for left camera view situated at right side of the virtual space, and vice versa. Hence we apply a mask which keeps the left side of left camera view and the right side of right camera view. The stitch equation is:

$$I(x,y) = M_1(x,y)I_1(x,y) + M_2(x,y)I_2(x,y) \tag{3.3}$$

$$M_1(x,y) + M_2(x,y) = 1 \tag{3.4}$$

Where I is stitched image, I_1 is image of first camera, I_2 is image of second camera, M_1 is mask image which applies on I_1, and M_2 is mask image which applies on I_2. The result is shown in Figure 3.10.

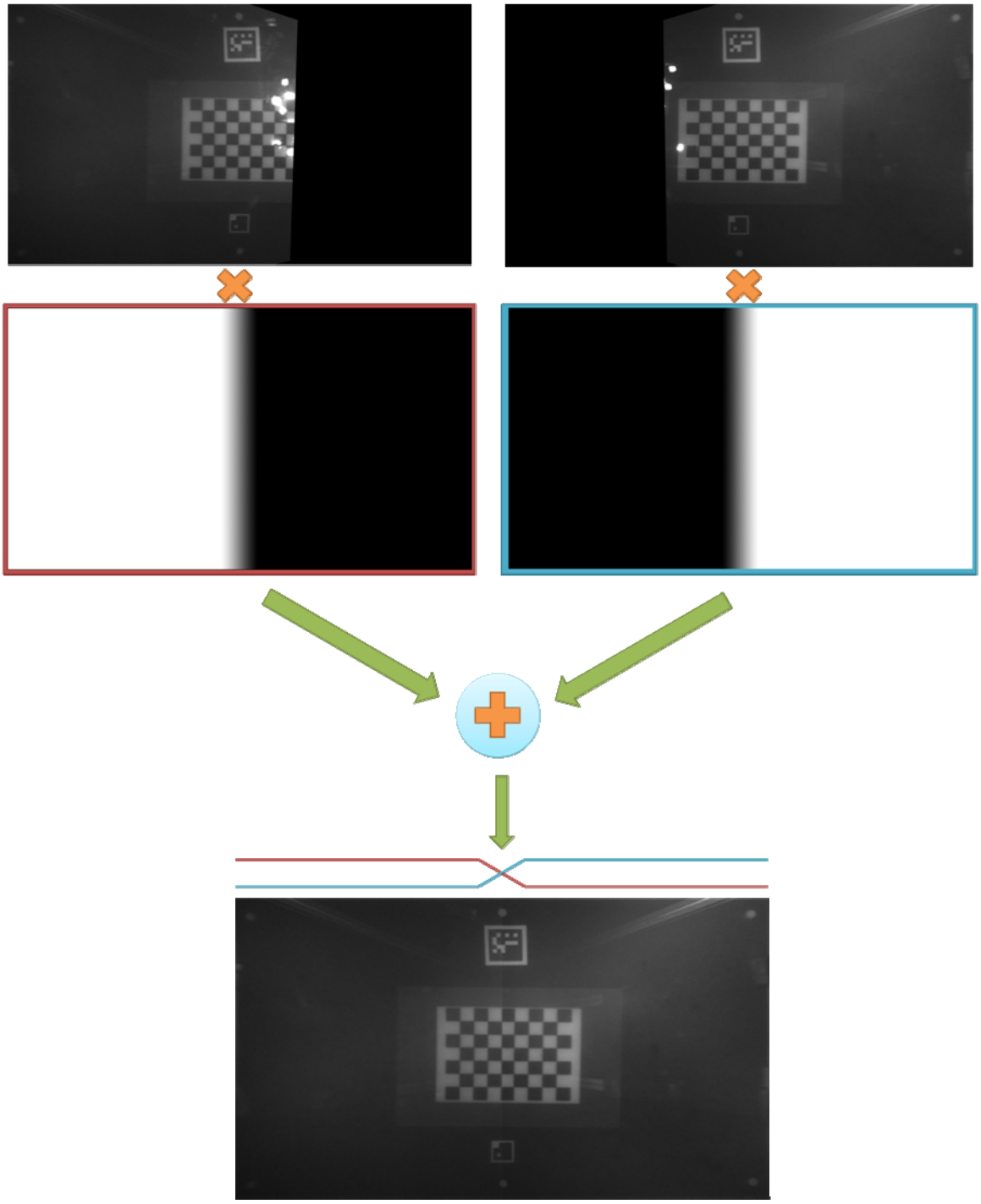


Figure 3.10: The flowchart of Image fusion.

Images fused by applying individual mask on each capture image. The weight of each pixel is representing by intensity which is normalized to dynamic range of zero to one. The red line and blue line present how these two capture images stitch together according to the weight.

3.2.2 Finger Detection

Considering a user puts hands on the digital surface, the contact areas of the hands would leave strong reflection, while the other parts dilute with distance away the surface. As a result, the intensity is much solid inside the contact areas, declines rapidly on the boundaries, and finally, shown smooth for the rest. We develop our multi-touch detection system by using this knowledge.

Figure 3.11: The flowchart of finger detection algorithm in TouchLib

Before introducing our finger detection algorithm, we have a quick review of TouchLib (Figure 3.11). At first, potential areas are extracted by applying background subtraction. Then simple highpass is used for leaving only blobs. Applying simple highpass will result in low intensity for each pixel including foreground object. Besides, in order to make picking an appropriate threshold in coming step easy, it is followed up by scaling image which stretches the intensity variance among pixels by square the value and magnifies intensity by multiply a scale. After that, they utilize a threshold to binarize image. Last but not least, finger analysis decides whether the foreground blob is a finger or not by checking if the blob fits the size of finger. As mentioned before, TouchLib suffered from non-uniform lighting problem. Therefore, we propose a finger detection algorithm referencing TouchLib to improve their drawbacks.

Figure 3.12: The flowchart of finger detection algorithm in our system.

Figure 3.12 is the processing procedures in our algorithm. First, potential areas are extracted by applying background subtraction. Second, a normalization operation is used to normalize each pixel from the subtracted images. The normalized foreground is then applied by a difference of Gaussian to leave blobs. After that, Gaussian smooth will make nearby fragments come together. Follow up by thresholding to binarize image. After several processing, finger analysis will decide whether the foreground blob is a finger or not.

Notice that our algorithm differs from their algorithm in two ways. First, we add normalization procedure and remove scale procedure. Second, we use different of Gaussian to replace simple highpass. We will describe the reasons in the following section.

We first extract potential areas by applying background subtraction. Due to the digital noise effect, our background image is built up by averaging first twenty images. There is no specific average number for generating background image depending on user's decision. There is an additional step in this image processing filter which is subtracting a certain value from potential area. We intended to filter unwanted foreground such as noise or palm by this preliminary thresholding.

Due to the non-uniform lighting condition, finger touch response differs from various positions among tabletop. In order to make finger detection uncomplicated, we

unify finger touch response through normalization. Notice that scaling process is removed from the algorithm owing to employing this procedure.

In our method, we model distribution of IR illumination, using specific material to simulate foreground and calculating each pixel's dynamic range through simulated foreground and background image. Then normalizing foreground by stretch each pixel's dynamic range to 0~255 which is the range of grayscale intensity. We generate a normal map based on dynamic range of each pixel. This normal map contains normal factor of each pixels. As a result, for incoming images, we normalize each pixel of foreground by multiply them with corresponding factors in normal map. The normal map construction equation is:

$$N(x,y) = \frac{255}{I_f(x,y) - I_b(x,y)} \tag{3.5}$$

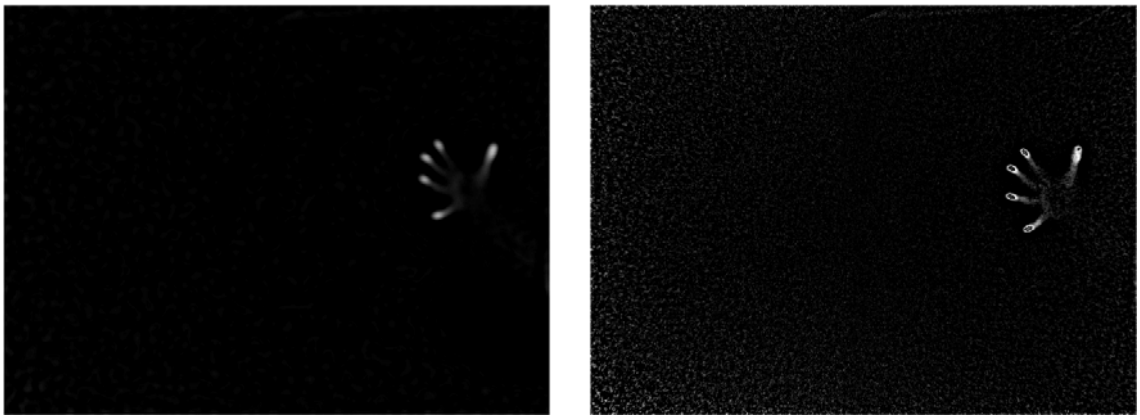
and the mapping equation is:

$$I'(x,y) = N(x,y) \times I(x,y) \tag{3.6}$$

where N means a normal map, I_f is a simulated foreground image, I_b is background image, I is an incoming image, and I' is a normalized image.

This procedure is mainly for remove noises and leave fingertips. This procedure differs from simple highpass of TouchLib. Simple highpass is to subtract the blurred image form the original image. But there are many noises in original image. If we subtract the blurred image from the original image, little noises still remain. The influence becomes serious especially while adopting normalization procedure(Figure 3.13). Hence we smooth the original image in the sake of removing noise, then subtract

the blurred image from it.



(a) After applying Difference of Gaussian (b) After applying Simple Highpass

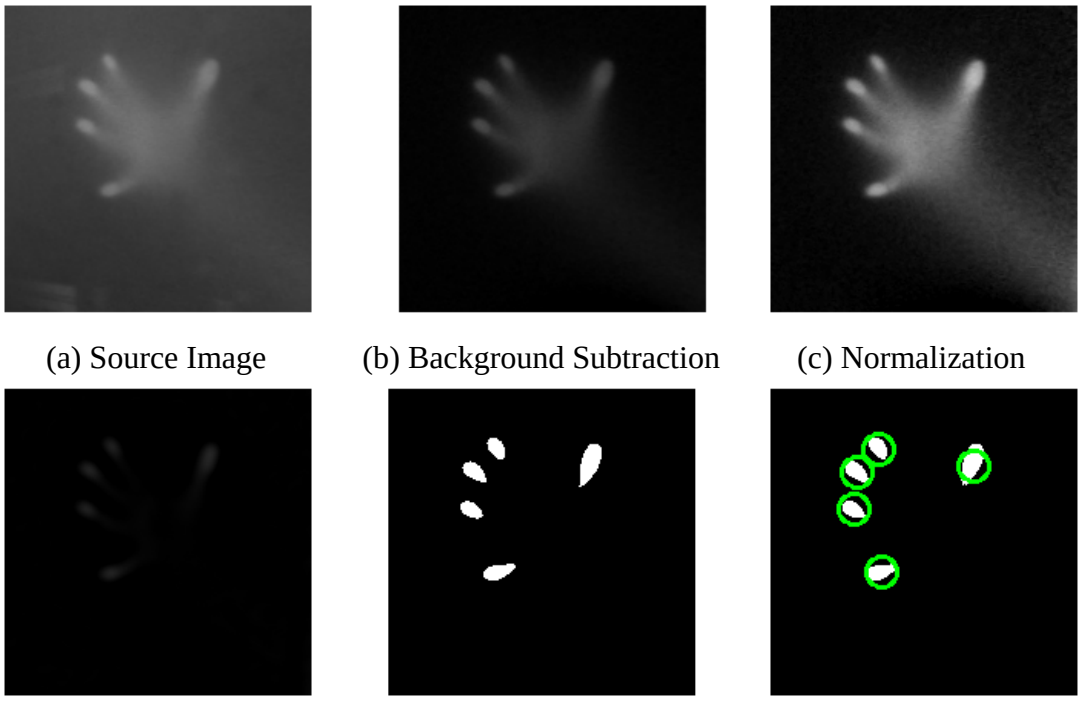
Figure 3.13: The difference between DoG and Simple Highpass

These two image shows the difference of applying Difference of Gaussian and Simple Highpass. We scale them by multiply five to let the effect be seen clearly.

Difference of Gaussians (DoG) is a grayscale image enhancement algorithm that involves the subtraction of one blurred version of an original grayscale image from another, less blurred version of the original. The blurred images are obtained by convolving the original grayscale image with Gaussian kernels having differing standard deviations. Blurring an image using a Gaussian kernel suppresses only high-frequency spatial information. Subtracting one image from the other preserves spatial information that lies between the ranges of frequencies that are preserved in the two blurred images. The difference of Gaussians discards all but a handful of spatial frequencies that are present in the original grayscale image. Thus, we adopt this procedure to remove noises and leave information in certain frequencies. To be specific, in finger detection the certain frequencies indicates fingertips.

This procedure is simply make current grayscale image into binary image for later usage. If intensity of the pixel exceed threshold, then the intensity will be set to 255, instead, it will be set to 0. After this process, we will have a binarized image only containing intensity of 0 or 255 for each pixel.

After previous processing, image is clean enough, leaving only blobs. The only thing we need to do is to employ connected component and verify whether they are fingers or not. Connected component can help us combine these binary data into fingertip candidates. In the end, we just need to decide if these candidates are fingers based on its size. The reason is that size of fingertips is in a specific range. If the size is too small, then it may be noises, instead, if it is too large, it may derive from palms or arms.



(a) Source Image (b) Background Subtraction (c) Normalization

(d) Difference of Gaussian (e) Binary (f) Finger Anaysis

Figure 3.14: The images produced after processing each procedure.

(a)(b)(c)(d)(e)(f) are images after adopting the procedure.

3.2.3 Finger Tracking

The main intention for adopting finger tracking is to smooth the trajectory of finger and insert fake finger while finger misses. No matter how stable the finger detection algorithm is, there still exist miss while recognizing fingertip. User may manipulate interactive application with rapid movement of finger. As a result, fingertip shown on captured image is blurred which result in various appearance from ordinary fingertip. Therefore, it may not be recognized as finger. Since phenomenon of missing fingertip occurs, we should insert fingers between recognized fingers among consecutive frames avoiding ambiguous operation happened.

We use Kalman filter [17] technique to track the position and velocity for fingertips. There are two main steps for applying Kalman Filter:

(1) Predicting the new state and its uncertainty.

(2) Correcting with the new measurement.

In our system, we assume the noise is white noise and the foreground fingertips move in a uniform velocity, so the state equation is:

$$\begin{pmatrix} x_{t+1} \\ vx_{t+1} \\ y_{t+1} \\ vy_{t+1} \end{pmatrix} = \begin{pmatrix} 1\ 1\ 0\ 0 \\ 0\ 1\ 0\ 0 \\ 0\ 0\ 1\ 1 \\ 0\ 0\ 0\ 1 \end{pmatrix} \begin{pmatrix} x_t \\ vx_t \\ y_t \\ vy_t \end{pmatrix} + \begin{pmatrix} wx_t \\ wvx_t \\ wy_t \\ xvy_t \end{pmatrix} \tag{3.7}$$

and the measurement equation is:

$$\begin{pmatrix} x_{t+1} \\ vx_{t+1} \\ y_{t+1} \\ vy_{t+1} \end{pmatrix} = \begin{pmatrix} 1\ 0\ 0\ 0 \\ 0\ 0\ 0\ 0 \\ 0\ 0\ 1\ 0 \\ 0\ 0\ 0\ 0 \end{pmatrix} \begin{pmatrix} x_t \\ vx_t \\ y_t \\ vy_t \end{pmatrix} + \begin{pmatrix} zx_t \\ zvx_t \\ zy_t \\ zvy_t \end{pmatrix} \tag{3.8}$$

where x and y stands for the 2D position, vx and vy stands for the velocity, t and $t+1$ is time index, w is state transition error covariance, and z is measurement error covariance.

After applying Kalman filter, the estimated foreground object's trajectory is smoothed. A simple strategy is used to assign detection results (observations) to Kalman trackers. After detection phase finished, each observation creates a searching area to find the nearest tracker according the distance between the observation and the trackers' position predictions. If no tracker is found in its searching area, the observation then creates itself a Kalman tracker. Trackers with no observation fed more than several consequent frames are discarded. With the high detection frames per second, the simple strategy works well.

3.2.4 Parameter Determination

In this section, we are going to present how we determine an applicable set of parameters for multi-touch detection system.

For a multi-touch detection system, an applicable set of parameters indicates applying these parameters on image processing filters of the detection algorithm which can perform a sensitive and noise-free finger detection system.

An ideal finger detection system consists of two principles, sensitive and noise-free. While users are manipulating multi-touch interactive surface, they expect their each movement will bring a correspondent reaction. If there is no action after touching the tabletop, users may feel frustrated and consider the system doesn't work well. Hence, sensitive is an essential requirement for an ideal finger detection system. On the other hand, noise indicates detecting fingertips while there is no actual touched finger. This phenomenon will lead to unexpected response depends on interactive application employed on multi-touch finger detection system. As minor as appearing fake fingers on the application, but severely it will affect actual operation. Consequently, noise free is also a vital requirement.

After defining the characteristics of an applicable set of parameters, we are going to introduce how we evaluate each parameter combination candidate for multi-touch detection system to fulfill the objective, sensitive and noise-free.

For detection systems, sensitivity represents low miss rate and noise-free means few false alarm. Hence, our measurement is to count number of miss and number of false alarm in a test set. The parameter combination containing least number of miss and false alarm while adopting on finger detection system is regarded as applicable set of parameters. We introduce details in the following paragraph.

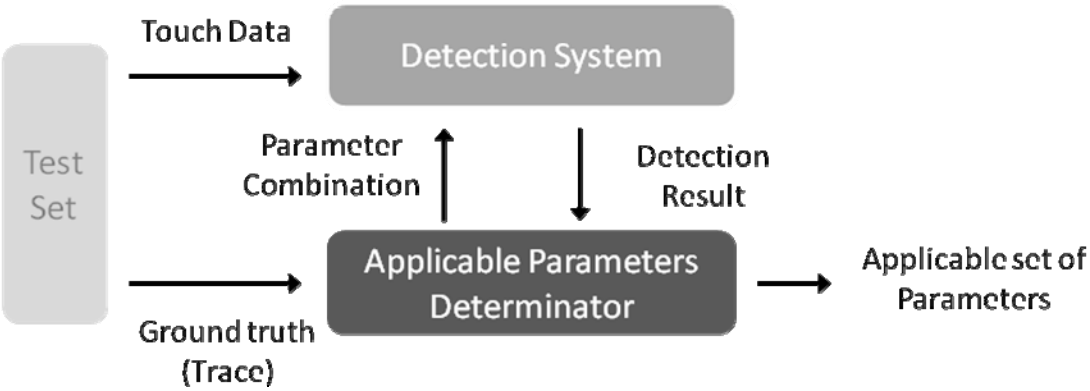


Figure 3.15: The schematic of optimizing parameters

Figure 3.15 is an overview of this applicable parameters search method. Applicable parameters determinator is the key component; it generates parameter combinations for detection system and evaluates each detection result to decide which combination results in best detection performance. We present this flowchart step by step to clarify our concept. First step is to generate parameter combination for detection system. Second step is detecting each touch data in test set, gathering each detection result for later usage. Third step is to compare ground truth and detection result, calculating number of miss and false alarm of each touch data. Last but not least, select an applicable set of parameters combination which has best detection performance.

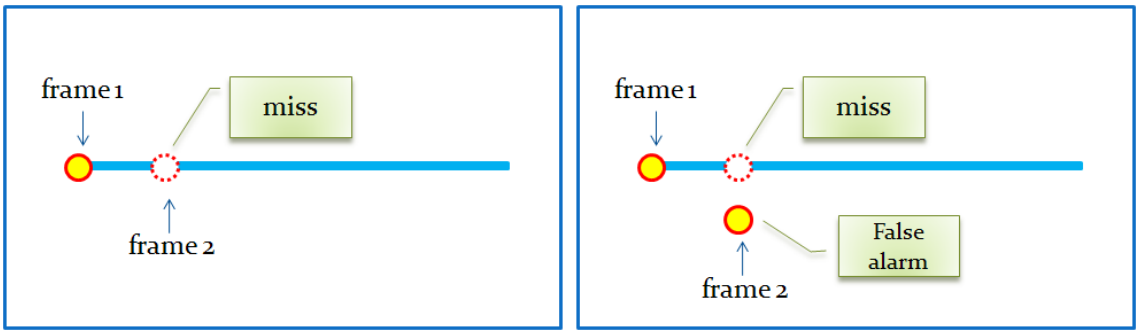
In the previous section, we had mentioned that an ideal finger detection system consists of two essential requirements, sensitive and noise-free. As a result, calculating number of miss and false alarm is an important feature in the optimal parameter search framework. Since our test set is a series of continuous frames starting from capturing images of user depicting head of the trace and ending up with reaching tail of the trace. We suppose that each frame should exist a finger landing on trace and fingers among frames should be continuous. Hence, there is a miss if no detected finger or there is a detected finger but not landing on trace (Figure 3.16). If the detected finger appears on

inappropriate position which doesn't satisfy the characteristic of continuality of fingers among successive frames (

(a) The correct location of detected finger (b) Uncontinuous detected finger

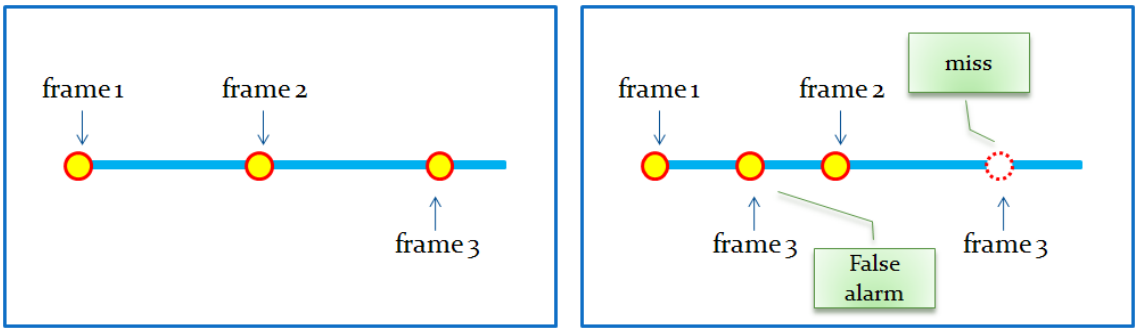
Figure 3.17), it will be regarded as false alarm. In addition, if there is more than one fingertip detected in one frame (Figure 3.18), they will be counted as false alarms, too.

For each parameter combination set, we sum up number of miss and false alarm in each frame of test set. Our objective is to minimize this number by determining an applicable set of parameters.



(a) No detected finger in the frame (b) Detected finger doesn't land on trace

Figure 3.16: Explanation for occurrence of miss and false alarm.



(a) The correct location of detected finger (b) Uncontinuous detected finger

Figure 3.17: Explanation for occurrence of errors and the sketch of uncontinuity

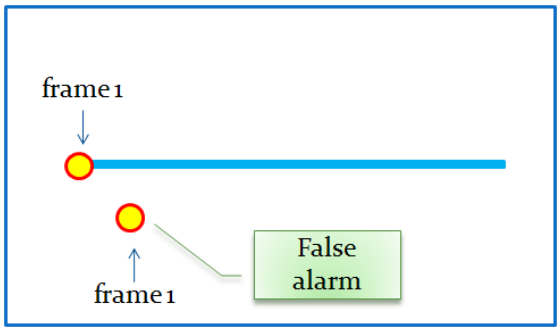


Figure 3.18: Explanation of occurrence of false alarm due to multiple detected fingers

In conclusion, we determine whether the parameters are an applicable set of parameter based on the requirements of an ideal multi-touch detection system, sensitive and noise-free. We propose a parameters evaluation method to fulfill these objective, and to experiment it by implementing two approaches in next chapter.

Chapter 4

Experiments

In this chapter, we two different kinds of experiments will be shown. The first one is the performance evaluation of our multi-touch finger detection system. The second one is to find an applicable set of parameters for our system.

To demonstrate the effectiveness of the algorithm, we include an evaluation on frames per second versus number of fingers simultaneously sliding on the surface. Our experiment was done on a Core2 Quad 2.66GHz and 2.67GHz machine with 4GB of memory. The video stream from the infrared camera is processed at 640x480 pixel resolution. In the Figure 4.1, the resulting curve shows a sub-linear relationship between frames per second and number of fingers simultaneously sliding on the surface. From Figure 4.1, it reveals that TouchLib can only achieve 30 frames per second. In addition, if there exist more fingers concurrently, the frame rate drops rapidly. In our system, the actual frame rate can achieve more than 60 frames per second. But it is limited to the frame rate of IR camera. It is obvious that performance of our system is much better than TouchLib.

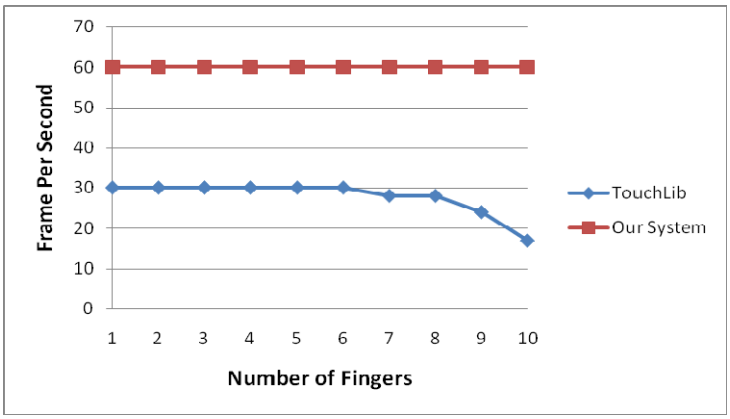


Figure 4.1: Performance evaluation of multi-touch detection system.

The average frames per second versus the number of fingers moving on the surface of TouchLib and Our system.

The following are how we search an applicable set of parameters for our multi-touch detection system. We start from how we collect touch data for test set. Follow up by deciding which parameters are required to be determined in our multi-touch detection system. Then we propose two sampling-based parameter search techniques to determine an applicable set of parameters.

(1) Data Collection

We collect touch data by asking user depicting the trace shown on tabletop. There are two reasons. First, depicting trace is an identical behavior as manipulating multi-touch interactive application. By gathering this kind of dynamic touch data, finger detection result of the optimal parameters applying on finger detection system is stable. Second, we can collect bunch of data in a short time.

In our experience, we had collected static data, but the parameters found by these test set doesn't work well on real operation. The way we gathering static data is to capture images while user touches the cross shown on different position of tabletop and to collect images while user's finger is nearest to the same cross. We determine

applicable set of parameters by differentiate real touch and almost touch. But there are two drawbacks. First one is that static data can't thoroughly represent user's behavior of manipulating multi-touch system due to most of real manipulation is dynamic movement instead of static touch. Second one is that it takes time to collect sufficient data.

Here we describe how to determine an appropriate trace for user to depict. We adopt a simplest trace – a horizontal line from left to right in the center of tabletop. The reason is that in our system, a horizontal line pass through regions of different lighting condition which sufficiently representing touched finger response on non-uniform lighting tabletop. Although there may exist other regions of various lighting condition which doesn't cover by our system currently. Since it's a prototype, we just propose our concept. We can definitely extend traces and collect more touch data in the future.

(2) Parameters decision

Recalling our finger detection algorithm, our image processing filters contains background subtraction, normalization, difference of gaussian, gaussian smooth, binary, and finger analysis.

4.1: The flowchart of finger detection in our system

Background subtracion consists of a parameter which is subtract value (s) from images after employing background subtraction.

Normalizaion is a process only need a normal map which is built up in advance. Hence, we don't need to adjust any parameter in this process.

Difference of Gaussian contains three parameters, standard deviation (σ_1) and smooth kernel (k_1) of less blurred image and the the other smooth kernel (k_2) of blurred image. Because we just want to blur original image a little bit to remove noise. According to our experience, set σ_1 to 3.78 and k_1 to 3x3 will bring good result. Therefore, we just need to adjust smooth kenel (k_2) of blurred image.

Binary contains one parameter, a threshold (t) for binarizing image.

Finger analysis consists of on parameter, finger size (f).

To sum up, there are four parameters need to be determined, subtract value (s),smooth kernel (k_2),threshold (t), and finger size (f).

(3) Experiments

Here we implement two sampling-based parameter search techniques to determine an applicable set of parameters.

Exhaustive Search

The first sampling-based parameter search technique is exhaustive search. This is a simplest way to achieve our goal. No matter how many parameters in a finger detection system, we exhaustively enumerate each parameter based on its range and step. Each parameter will have a correspondent number of miss and false alarm while applying them on detecting gathered test set. In the end, we select an applicable set of parameters from all enumerated parameter combinations based on our intention-minimizing number

of miss and false alarm.

In order to reduce time spend on searching an applicable set of parameters, we define range and step for each parameter in the experiment (Table 4.1). We enumerated 1296 ($=6\times6\times6\times6$) sample. In this case, there exist 244 parameter combinations that satisfied our parameter evaluation requirements, number of miss and number of false alarm are zero. We pick up one of them applying on our multi-touch detection system which works well while employing operation on tabletop.

	Subtract value	Smooth kernel	Threshold	Finger size
Low bound	0	7	10	15
Step	3	2	3	9
High bound	15	17	25	60

Table 4.1: Range and step for each parameter in the experiment

In his experiments, since our data collection contains only horizontal line which is the simplest case, therefore, there exist multiple applicable sets of parameters which have zero miss and zero false alarm. That means although we can find an applicable parameter combination, it may far from optimal set of parameter for our multi-touch detection system due to indiscriminate data.

Particle Filtering

The second sampling-based parameter search technique is based on particle filtering. Searching an applicable set of parameter is not a linear problem because it is highly variation among different parameter combinations. That is, we can’t find a linear transformation between parameter space and finger detection result. Parameter combinations are complicated space. In addition, there exists multiple Gaussian

distribution among this space which means there may be multiple optimal solutions.

Particle filtering [16] is a Bayesian sequential factored sampling approach, which is generally applied to solve non-linear estimation problems. Therefore, we adopt the this approach to accomplish applicable parameter searching.

Before we start to explain how we utilize it, let us have a quick view of particle filtering. The key idea is to represent the required posterior function by a set of random samples with associated weights and to compute estimates based on these samples and weights. The fundamental concept is clearly expressed in Figure 4.2. The most crucial part in particle filtering is how to evaluate the particles by calculating the likelihood function $P(Z_t \mid X_t)$ which is measured by observation. The transition probability function used for determining how the particles transformed in next iteration is important, too. But generally, the transition probability is set as a normal distribution. Therefore, we will introduce how we utilize particle filtering by defining the likelihood function.

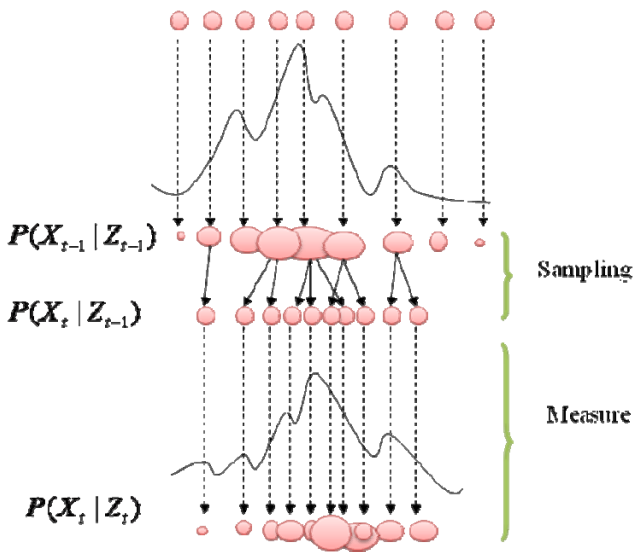


Figure 4.2: The schematic view of particle filtering algorithm.

We measure our likelihood by defining energy function for our system first.

E represents the energy function accounting for penalty of finger detection which is error that we are going to minimize. It is defined as:

$$E = \sum_{i \in T} \left((\text{\# of miss})_i + (\text{\# of false alarm})_i \right) \tag{4.1}$$

Where T stands for test set, and i is the index of touch data in the test set. The likelihood is defined as:

$$P(Z_t \mid X_t) \propto \exp\left(-E^2 / 2\sigma^2\right) \tag{4.2}$$

where X_t is a parameter combination which will result in finger detection result by applying it on finger detection system, Z_t is ground truth of finger detection result, and σ is a standard deviation. The likelihood is measured by comparing ground truth and finger detection result generated by applying parameter combination on finger detection system.

The following are how we adopt the particle filtering (Figure 4.3). First, we uniform sample particles in the dynamic range of each parameter. Each particle is high-dimension which depends on how many parameters we are going to determine in the finger detection algorithm. After that, we will apply the parameter combination comes from particle to weight particles respectively by measuring likelihood. Then we normalize each particle to make the sum of probabilities to one. Next, if the number of miss and false alarm is equal to zero which indicates the parameter combination is good enough that no errors occurs during finger detection in the test set, we output those parameters as applicable parameter combination. If not, we check if number of iterations over limited iteration determine by us subsequently. If yes, the procedure terminates and we will output a parameter combination which has fewest miss and false alarm among all parameter combinations. If not, we resample particles for next iteration

depending on the weight of current particles. We sample more particles from particle with higher weight in current iteration. The sampling method is individually sampling each parameter based on Gaussian distribution. Then every new particle, namely new parameter combination, is composed of each new sampled parameter. We will obtain an applicable set of parameter for our multi-touch finger detection system in the end. In this case, we determined range of each parameter for the first uniform sampling and the mean and variance of Gaussian distributions for each parameter while resampling. Here for consistence, the range of each parameter is equal to the range decided in previous sampling method. Mean is zero, because we want to sample each parameter based on value of current particles, and the variance is equal to the step in previous sampling method. In this experiment, we obtain an applicable set of parameters at first iteration which also works well in real manipulation.

The goal of this sampling technique is to obtain an applicable set of parameters in a shorter time than exhaustive search. But in this experiment, since there exist multiple applicable sets of parameters motioned in previous experiment, the advantage of utilizing this sampling-based parameter search instead of random sampling is unobvious.

In conclusion, it’s easy to find an applicable set of parameters while using current test set. But if the format of data increases, for instance, adding samples with various finger sizes, collecting data with more difficult trace and so on, we may obtain a set of parameters which works better than current found applicable set of parameters.

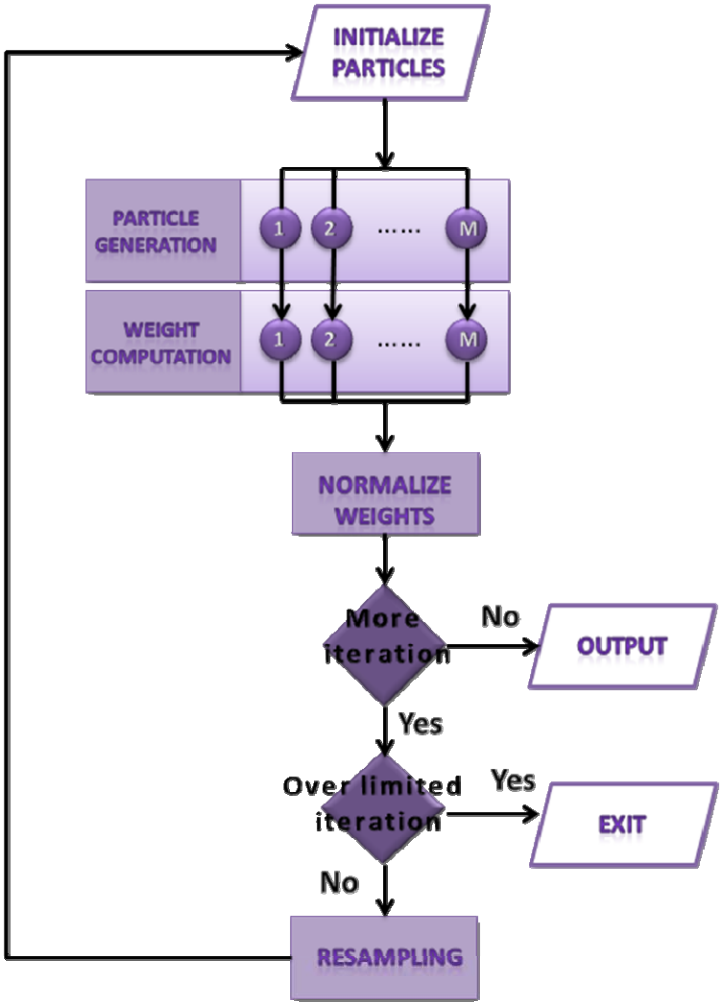


Figure 4.3: The diagram of particle sampling applied on our system.

For detection systems, sensitivity represents low miss rate and noise-free means few false alarm. Parameter evaluation method includes three parts. First, we have to decide how we collect touch data which are used for test set. Second, we need to determine how we measure each parameter combination candidate for finger detection system. Then we propose two sampling-based parameter search techniques to determine an applicable set of parameters.

Chapter 5

Conclusion and Future Work

5.1 Conclusion

In this work, we develop a GPU-accelerated multi-touch detection which solves the problem of non-uniform lighting condition of tabletop system. Since we have a high frame-rate finger detection system, the reactions of interactive application will response immediately. Hence, user may feel comfortable due to smooth operation.In addition, we don’t have to worry about how to generate a uniform lighting distribution while adopting large scale display anymore. Besides, implementation of our multi-touch detection system includes merging multiple cameras into a virtual camera, which can be compatible with existent finger detection system. In addition, we propose a sampling-based parameter search method to help developer determine an applicable set of parameters for multi-touch detection system.

5.2 Future Work

Possible directions of our finger detection for multi-touch tabletop display system might includes improving method of collecting touch data used for parameter determination and optimizing parameter for multi-touch detection system.

First, method of collecting samples can be more intelligent. We can collect sample and evaluate the touch data at the concurrently. Meanwhile, automatically generate trace,

which is used for collecting sample, guiding user toward positions where the sample data is not sufficient enough to represent the finger touch response.

Second, we can estimate optimal parameter combination for multi-touch detection system instead of simply determining an applicable set of parameters.

Bibliography

[1] P. Dietz and D. Leigh, “DiamondTouch: a multi-user touch technology,” in *UIST '01: Proceedings of the 14th annual ACM symposium on User interface software and technology*. New York, NY, USA: ACM Press, 2001, pp. 219-226.

[2] “Synaptics Touchpad.” [Online]. Available: http://www.synaptics.com

[3] G. Blasko and S. Feiner, “Single-handed Interaction Techniques for Multiple Pressure-Sensitive Strips,” in CHI ’04: *Proceedings of the SIGCHI conference on Human factors in computing Systems.* New York, NY, USA: ACM Press, 2004, pp.1461-1464

[4] “Apple iPhone.” [Online]. Available: http://www.apple.com/iphone/

[5] J. Y. Han, “Low-cost multi-touch sensing through frustrated total internal reflection,” in UIST '05: *Proceedings of the 18th annual ACM symposium on User interface software and technology*. New York, NY, USA: ACM Press, 2005, pp. 115-118.

[6] J. Rekimoto and N. Matsushita, “Perceptual surfaces: Towards a human and object sensitive interactive display,” in PUI ’97: *Proceedings of ACM Workshop on Perceptual User Interfaces.* 1997.

[7] “Microsoft surface.” [Online]. Available: http://www.microsoft.com/surface/

[8] M. Kaltenbrunner and R. Bencina, “Reacvision: a computer-vision framework for table-based tangible interaction,” in TEI ’07: *Proceedings of the 1st international conference on Tangible and embedded interaction*. New York, NY, USA: ACM, 2007, pp 69-74

[9] C. von Hardenberg and F. Berard, “Bare-hand human-computer interaction," in

PUI ’01: *Proceedings of the 2001 workshop on Perceptive user interfaces*. New York, NY, USA: ACM Press, 2001, pp. 1-8

[10] A. D. Wilson, “Playanywhere: a compact interactive tabletop projection-vision system," in UIST ’05: *Proceedings of the 18th annual ACM symposium on User interface software and technology*. New York, NY, USA: ACM Press, 2005, pp. 83-92.

[11] A. D. Wilson, “Touchlight: an imaging touch screen and display for gesture-based interaction," in ICMI ’04: *Proceedings of the 6th international conference on Multimodal interfaces*. New York, NY, USA: ACM Press, 2004, pp. 69-76.

[12] S. Malik, A. Ranjan, and R. Balakrishnan, ”Interacting with large displays from a distance with vision-tracked multi-finger gestural input," in UIST ’05: *Proceedings of the 18th annual ACM symposium on User interface software and technology*. New York, NY, USA: ACM Press, 2005, pp. 43-52.

[13] S. Malik and J. Laszlo, “Visual touchpad: a two-handed gestural input device,” in ICMI ’04: *Proceedings of the 6th international conference on Multimodal interfaces.*New York, NY, USA: ACM Press, 2004, pp. 289-296.

[14] L.-W. Chan, Y.-F. Chuang, Y. wei Chia, Y.-p. Hung, and J.Y. jen Hsu, “A new method for multi-finger detection using a regular diffuser,” in HCI *(3)*, 2007, pp.573-582

[15] “TouchLib.” [Online]. Available: http://www.nuigroup.com/touchlib/

[16] M. Isard and A. Blake, “Condensation conditional density propagation for visual tracking,” in IJCV ’98: *International Journal of Computer Vision*. 1998, pp. 5-28.

[17] G. Welch and G. Bishop, “An introduction to the Kalman filter,” in SIGGRAPH ’01: *Special Interest Group on Graphics and Interactive Techniques*. Los Angeles, CA, USA, ACM Press

[18] "Microsoft DirectShow." [Online]. Available:

http://msdn.microsoft.com/en-us/library/dd390351(VS.85).aspx